

Growing and Shrinking a Linked List

Insertion and Deletion — Front, End, and Anywhere Between

Unit II • Lectures 17-18

Position	INSERT	DELETE
Front	$O(1)$	$O(1)$
End	$O(n)$	$O(n)$
Position	$O(n) + O(1)$	$O(n) + O(1)$

The big idea: delete is insert, in reverse

Lecture 17 learned to ADD a node at three positions. Today we REMOVE a node from the same three positions — and the plan for finding each spot is identical.

- Front: still needs zero traversal.
- End: still needs a walk — but delete's walk stops ONE NODE EARLIER than insert's did. That is today's one new trap.
- Position: still find-the-node-before, then rewire — except now you FREE a node instead of creating one.

FRONT: $O(1)$ both ways

insertFront — add

```
node->next = head;  
return node;
```

New node points at old head, then BECOMES head.

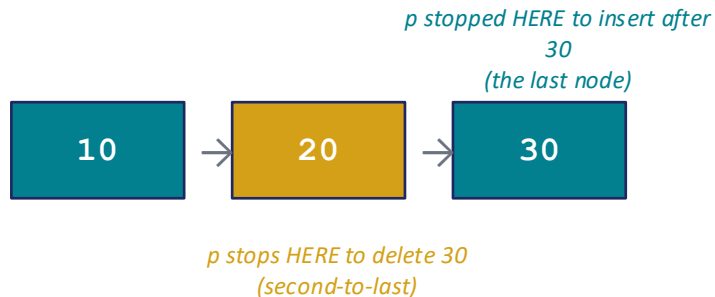
deleteFront — remove

```
temp = head;  
head = head->next;  
free(temp);
```

SAVE old head, MOVE head, then free the saved node.

- Both touch ONLY head — no traversal, no walking.
- Delete's extra step: you must SAVE the old head before moving head, or its address is lost and it can never be freed — the same save-before-free discipline as linked-stack pop.
- Underflow check: deleteFront must test head == NULL FIRST — there is nothing to delete from an empty list.

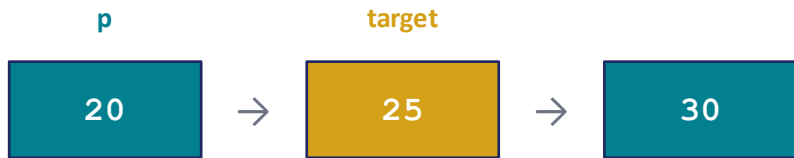
END: the walk that stops one node earlier



```
/* insertEnd (L17) */  
while (p->next != NULL)  
    p = p->next;  
p->next = node;  
  
/* deleteEnd (today) */  
while (p->next->next != NULL)  
    p = p->next;  
free(p->next);  
p->next = NULL;
```

One extra `->next` in the while-condition is the ENTIRE difference.

POSITION: unlink instead of link



```
/* insertAtPosition (L17) */
node->next = p->next;
p->next    = node;

/* deleteAtPosition (today) */
target     = p->next;
p->next     = target->next;
free(target);
```

- Insert: p reaches around a NEW node into the list.
- Delete: p reaches around an EXISTING node, cutting it out — then that node is freed.
- Same 'find the node before the target' plan both times; only the last step differs (attach vs. free).

Underflow — and why "overflow" barely applies here

Underflow — real and important

head == NULL means there is nothing to delete. EVERY deletion function must check this FIRST, exactly like stack/queue underflow.

Overflow — barely applies

A linked list has no MAX, so there is no array-style overflow. The only failure left is malloc returning NULL — and that can only happen during INSERT, never delete.

- Every insert function checks malloc's result; every delete function checks head == NULL. They guard against DIFFERENT failures because they do different jobs.
- This is a good moment to notice: syllabus headings written for arrays (overflow/underflow) do not always map perfectly onto linked structures — worth questioning, not just memorising.

Full trace: insert five, then delete four ways

Call	List afterward
<code>insertEnd x5: 10,20,30,40,50</code>	<code>10 -> 20 -> 30 -> 40 -> 50 -> NULL</code>
<code>deleteFront()</code>	<code>20 -> 30 -> 40 -> 50 -> NULL</code>
<code>deleteEnd()</code>	<code>20 -> 30 -> 40 -> NULL</code>
<code>deleteAtPosition(2)</code>	<code>20 -> 40 -> NULL</code>
<code>deleteFront() x2</code>	<code>list is empty</code>
<code>deleteFront() again</code>	<code>Underflow: list is empty</code>

Insert vs Delete, every position, one table

Position	Insert cost	Delete cost	Delete's extra step
Front	$O(1)$	$O(1)$	Save old head before moving it
End	$O(n)$	$O(n)$	Stop ONE node earlier (2nd-to-last)
Position k	$O(n)$ find + $O(1)$ link	$O(n)$ find + $O(1)$ unlink	free() the removed node

The cost of REACHING a position is identical for insert and delete. Only the last step differs: attach a new node, or free an old one.

Insert and delete: the same map, opposite direction

- Front: $O(1)$ both ways. Insert links then moves head; delete saves then moves then frees.
- End: $O(n)$ both ways, but delete's walk stops ONE node earlier than insert's — at the second-to-last node, not the last.
- Position k : $O(n)$ to find the node before the target, $O(1)$ to finish — attach (insert) or unlink-then-free (delete).
- Underflow (head == NULL) guards every delete. "Overflow" barely applies to a list — only malloc failure during insert comes close.
- The golden rule survives from Lecture 9: save/link BEFORE you move or free, or you lose the address you needed.

EXIT TICKET Starting from 5 -> 10 -> 15 -> 20 -> NULL, trace deleteAtPosition(head, 3) by hand. Which node is freed, and which two nodes end up next to each other?